LEHRSTUHL FÜR RECHNERARCHITEKTUR UND PARALLELE SYSTEME

Grundlagenpraktikum: Rechnerarchitektur

Praktikumsordnung für den Zweig ASM/Performanz
(Stand: 10. Juni 2024)

1. Grundsätzliches

Diese Praktikumsordnung soll einen ordnungsgemäßen Ablauf des Grundlagenpraktikums „Rechnerarchitektur“ (GRA) gewährleisten. Sie ist nicht Bestandteil der Prüfungsordnung und gibt auch keine Hinweise auf den fachlichen Inhalt des Praktikums.

2. Durchführungsform

2.1. Form des Praktikums

Das Praktikum wird in Form von Gruppenprojekten durchgeführt. In Vorbereitung auf die Projekte werden in der ersten Hälfte des Semesters im Praktikumsbetrieb und durch Hausaufgaben die notwendigen Grundlagen vermittelt. Die hier erarbeiteten Kenntnisse werden für die Projektaufgaben vorausgesetzt. Im zweiten Teil des Semesters bekommt jede Gruppe (bestehend aus 3 Personen) ein Projekt zugeteilt, welches in einer vorgegebenen Zeit zu bearbeiten ist. Für das erfolgreiche Bestehen des Praktikums sind eine Ausarbeitung, die Projektimplementierung und die Projektpräsentation Voraussetzung. Die Ausarbeitung und die Projektpräsentationen sind in *deutscher Sprache* anzufertigen bzw. vorzutragen.

2.2. Prüfungsvoraussetzungen und Notenbonus

Voraussetzung für die Teilnahme an der Projektphase ist das Erreichen von 50% der Hausaufgabenpunkte in der Vorbereitungsphase. Dies gewährleistet eine entsprechende Qualifikation aller Gruppenmitglieder in der Projektphase.

Studierende, die aufgrund ihrer Fachprüfungsordnung 8 ECTS für dieses Praktikum benötigen, müssen eine Zusatzaufgabe erfolgreich absolvieren (Abschnitt 2.6). Studierende, die nur 5 ECTS benötigen, sind von dieser Aufgabe befreit.

Bei Erreichen von mindestens 75% der Hausaufgabenpunkte gibt es einen Notenbonus (siehe Abschnitt 4.2).

2.3. Vorgehensweise

Die Zuordnung der Gruppen und Aufgaben erfolgt durch die Praktikumsleitung. Die Projekte werden in Git-Repositories für die jeweiligen Teams auf Artemis bereitgestellt und sind dort abzugeben. In den Repositories sind auch regelmäßig Zwischenstände zu

speichern, die eine eindeutige Zuordnung der Urheberschaft der verschiedenen Teile der Implementierung über das *Author*-Feld von Commits ermöglichen. Die dort vorgegebene Ordnerstruktur und erwarteten Dateinamen sind einzuhalten, insbesondere auch Groß- und Kleinschreibung. Für die Bewertung wird der letzte Commit des `main`-Branches zum Zeitpunkt der Abgabefrist herangezogen.

2.4. Dauer der Bearbeitung

Das Praktikum ist in einem Semester zu absolvieren. Mit Ausgabe der Projektaufgaben wird ein Abgabetermin festgelegt, zu welchem das Ergebnis eingereicht werden muss. Sollte diese Frist nicht eingehalten werden können, ist ein Ausnahmeantrag zu stellen. Bei diesem entscheidet die Praktikumsleitung in Rücksprache mit dem Tutor, ob eine Verlängerung möglich ist. Eine zweite Verlängerung der Frist ist nicht möglich. Sollte der Ausnahmeantrag nicht genehmigt und das Projekt trotzdem erst nach Fristende fertiggestellt werden, gilt das Praktikum als nicht bestanden.

2.5. Verhalten der Gruppenmitglieder

Es wird erwartet, dass jedes Gruppenmitglied an allen Teilen mitarbeitet und insbesondere *zu jedem Teil* Fragen beantworten kann. Die Gruppenmitglieder haben nachzuweisen, dass sie sich um die Integration aller Gruppenmitglieder bemüht haben. Sollten dennoch Gruppen mit weniger als drei Personen zustande kommen, so gelten die Ausnahmeordnungen unter Punkt 5.1.

2.6. Zusatzleistung für 8 ECTS

Studierende, die auf Grund Ihres Studienplans 8 ECTS benötigen, müssen in Einzelarbeit eine Zusatzaufgabe bearbeiten, die nicht Bestandteil der Projektarbeit ist. Abgabeform und Bearbeitungszeitraum der Zusatzaufgabe werden gesondert bekannt gegeben. Bewertet wird lediglich, ob die Anforderungen vollständig eingehalten wurden, eine darüber hinausgehende qualitative Bewertung erfolgt nicht. Studierende, die lediglich 5 ECTS benötigen, müssen die Zusatzaufgabe nicht bearbeiten.

3. Projektanforderungen

Das Praktikumsprojekt besteht aus zwei Teilen: Implementierung und Projektpräsentation.

3.1. Implementierung

Die in der Aufgabenstellung spezifizierten Algorithmen müssen gemäß den dort genannten Anforderungen implementiert werden.

In jedem Fall muss zusätzlich zur Hauptimplementierung mindestens eine weitere Vergleichsimplementierung mit abgegeben werden, die geeignet ist, die Qualität der

Hauptimplementierung zu evaluieren; wenn nötig sind auch mehrere Vergleichsimplementierungen zu erstellen.

Das Rahmenprogramm ist so zu erstellen, dass es sich an die Vorgaben der Aufgabenstellung hält. Die vorgegebenen Ein- und Ausgabeformate sind genau einzuhalten und fehlerhafte Eingaben sind mit einer sinnvollen Fehlermeldung zu behandeln. Für Fehlermeldungen ist stderr zu benutzen. Nichteinhalten der Angaben aus der Aufgabenstellung kann hier zu deutlichen Punktabzügen führen.

Zusammen mit dem Programm ist ein `Makefile` abzugeben, welches durch den Aufruf mit `make` die Abgabe zu *insgesamt einer* ausführbaren Binärdatei kompiliert, die insbesondere alle Performanztests und abgebenen Implementierungen enthält.

Sehr wichtig ist das sinnvolle Strukturieren und Kommentieren des Codes. Unzureichende oder nichtssagende Kommentare sowie eine schwer nachvollziehbare Struktur werden die Benotung negativ beeinflussen.

3.2. Projektbericht

Zudem ist zusammen mit dem Programm ein Projektbericht abzugeben, der Projekt, Implementierung, Ergebnisse und Arbeit der Teilnehmenden kurz beschreibt. Der Projektbericht ist in *deutscher Sprache* zu verfassen und am *am Abgabetermin des Projekts* abzugeben. Er darf maximal 400 Wörter lang sein und ist in Markdown zu verfassen. Der Kurzbericht soll nach einer möglichst kurzen Erklärung der Aufgabenstellung einen Überblick über die verschiedenen Implementierungen und die darin verwendeten Optimierungen geben. Des weiteren sollte die Methodik und Messunmgebung der Performanzmessungen hier kurz dokumentiert werden. Schlussendlich sollen die Ergebnisse des Benchmarkings und des Gesamtprojekts hier kurz zusammengefasst werden. Zusätzlich sollte pro Person ein kurzer Abschnitt verfasst werden, der den eigenen Anteil an der Projektarbeit kurz beschreibt.

3.3. Projektpräsentation

Im Anschluss an das Praktikum muss jede Gruppe einen Projektvortrag von 15 Minuten halten (hartes Zeitlimit!). In diesem sollen die wichtigen Aspekte des Projekts vorgestellt werden.
Folgende Bereiche sind in einem sinnvoll gewählten Umfang zu behandeln:

1. *Problemstellung*
   In diesem einleitenden Teil soll die Problemstellung eingeführt, beschrieben und gegebenenfalls analysiert und genauer spezifiziert werden.

2. *Lösungsansätze und Optimierungen*
   Der gewählte Lösungsansatz soll klar und nachvollziehbar beschrieben und analysiert werden. Wichtige Entscheidungen werden diskutiert und begründet, umgesetzte Optimierungen werden erläutert. Wo relevant beinhaltet dies eine Gegenüberstellung möglicher Lösungsalternativen und Überlegungen zu möglichen weiteren Optimierungen.

3. *Korrektheit* bzw. *Genauigkeit* (je nach Aufgabe)
   Die Korrektheit bzw. Genauigkeit (jeweils wo angemessen) *der Implementierung und des Ansatzes* soll gezeigt und bewertet werden. Nach Möglichkeit sind hierfür auch automatisierte Tests zu implementieren und abzugeben. Außerdem sollen *sinnvolle und repräsentative Beispiele* für Eingaben und die dazu berechneten Ergebnisse gezeigt werden.

4. *Performanzanalyse*
   In diesem Teil soll die Performanz der Implementierung analysiert und bewertet werden. Hierzu sind geeignete Methoden zu wählen, beispielsweise Zeitmessungen. Bei Zeitmessungen ist mindestens eine weitere zur Evaluierung der Implementierung geeignete Vergleichsimplementierung heranzuziehen und diese auf verschiedenen repräsentativen Eingaben auszuführen. Die gewonnenen Ergebnisse sind zu bewerten und Ursachen für beobachtete Performanzunterschiede zu benennen.

5. *Zusammenfassung und Ausblick*
   Neben einer kurzen Zusammenfassung des umgesetzten Projektes kann dieser Abschnitt einen kurzen Ausblick enthalten. In diesem wird beurteilt, wo zusätzliches Potential für Verbesserungen existiert, ob rückblickend eine andere Lösungsalternative besser wäre.

Es wird erwartet, dass der Vortrag in ungefähr gleichen Teilen von allen Gruppenmitgliedern gehalten wird. Der Vortragstermin wird durch die Praktikumsleitung zugeteilt. Die Folien der Abschlusspräsentation sind zusammen mit den anderen Dokumenten *am Abgabetermin des Projekts mit Seitenverhältnis 16:9* abzugeben. Es können während des Vortrags nur die abgegebenen Folien verwendet werden.
Im Rahmen des Vortrags können jedem Gruppenmitglied zu jedem Teil des Projekts Fragen gestellt werden. Zusätzlich werden die Praktikumsleitung, Lehrstuhlmitarbeitenden und die Tutoren die Möglichkeit haben, den Teilnehmenden Fragen zu den im Praktikum erlernten Inhalten, den verwendeten Hardware- und Software-Komponenten und der relevanten Theorie zu stellen.

4. Bewertung

Es werden nur Studierende benotet, deren Gruppe die Präsentation und den Projekt-Code in den erwarteten Formaten eingereicht und die Projektpräsentation gehalten haben. Die Tutoren und Projektbetreuer bewerten die Ergebnisse nach den in den Bewertungsbögen aufgeführten Kriterien. Der Notenbonus verbessert die Note um eine Notenstufe (0.3/0.4); eine Verbesserung der Noten 1.0, 4.3, 4.7 und 5.0 ist dabei ausgeschlossen.

4.1. Zusatzleistung für 8 ECTS

Bei Studierenden, die 8 ECTS benötigen (siehe Abschnitt 2.6) und die Zusatzaufgabe nicht bearbeitet haben oder die Anforderungen nicht vollständig erfüllt haben, wird das Praktikum unabhängig von der sonstigen Projektarbeit mit der Note 5.0 bewertet. Im Übrigen hat die Bearbeitung der Zusatzaufgabe keinen Einfluss auf die Bewertung.

4.2. Notenbildung

Aus den Bewertungskriterien ergeben sich Einzelbewertungen für Implementierung und Vortrag. Es müssen für *beide* dieser Einzelbewertungen mindestens 33% Punkte erreicht werden, andernfalls wird das Praktikum direkt mit der Note 5.0 bewertet. Die Gesamtpunktzahl ergibt sich aus der Summe dieser Einzelbewertungen. Der Notenbonus verbessert die Note um eine Notenstufe (0.3/0.4); eine Verbesserung der Noten 1.0, 4.3, 4.7 und 5.0 ist dabei ausgeschlossen.

4.3. Technische Probleme

Es ist bekannt, dass Technik auch ausfallen kann. Während wir uns bemühen, die Infrastruktur möglichst ununterbrochen bereitzustellen, können wir dies selbstverständlich nicht garantieren. Falls also Infrastruktur wie die Praktikumswebsite, Artemis oder die `lxhalle` kurz vor der Abgabe nicht verfügbar sein sollte, können wir das in der Bewertung nicht berücksichtigen. Es empfiehlt sich, entsprechende Puffer für solche Fälle einzuplanen.

4.4. Einsicht

Bei der Einsicht kann sich ein Studierender über die Zusammensetzung der Note informieren. Das Ziel der Einsicht ist die Nachvollziehbarkeit der Bewertung für die Studierenden. Spätestens mit Freigabe der Bewertung werden die Formalitäten der Einsicht bekannt gegeben. Die Einsicht kann auch online erfolgen, eine persönliche Einsicht muss durch die Praktikumsleitung nicht angeboten werden. Bei einer persönlichen Einsicht ist die Dauer pro Gruppe auf 15 Minuten beschränkt. Beanstandungen sind spätestens am letzten Einsichtstag einzureichen und werden im Anschluss überprüft. Eine Antwort durch die Praktikumsleitung auf die Beanstandung kann auch per E-Mail erfolgen. Durch eine etwaige Nachkorrektur kann es auch zu einer Verschlechterung der Bewertung kommen.

5. Sonderfälle

5.1. Gruppen aus weniger als drei Personen

Bei der Gruppenbildung ist darauf zu achten, dass keine Einzelpersonen übrig bleiben und es maximal viele Dreiergruppen gibt. Das bedeutet, dass eine Einzelperson und eine Dreiergruppe sich zusammen in zwei Zweiergruppen teilen.

Falls ein Gruppenmitglied während des Projekts die Gruppe verlässt oder sich trotz wiederholter Bemühungen der übrigen Gruppe nicht oder nur sehr geringfügig an der Projektarbeit beteiligt, sind der betreuende Tutor und die Praktikumsleitung *frühzeitig zu informieren*. Erst nach expliziter Feststellung durch die Praktikumsleitung gilt die Gruppe als verkleinert.

Sollte eine Gruppe aus weniger als drei Personen bestehen, und dies aus Gründen, die diese Personen nicht zu vertreten haben, so ist der Projektaufwand in Absprache mit den Tutoren und der Praktikumsleitung zu reduzieren, insbesondere durch eine Verkürzung der Vortragsdauer.

5.2. Abwesenheit am Vortragstermin

Sollte ein Gruppenmitglied am bekannt gegebenen Vortragstermin nicht anwesend sein, so wird von den übrigen Gruppenmitgliedern der Vortrag in normaler Länge gehalten. Bei Abwesenheit aus einem anerkannten Grund (z.B. nachgewiesene Krankheit) findet innerhalb von einem Monat, spätestens jedoch in der ersten Vorlesungswoche des folgenden Semesters, eine Einzelprüfung statt. Bei unbegründeter Abwesenheit gilt das Praktikum für den abwesenden Studierenden als nicht bestanden.

5.3. Täuschungsversuch

Die Praktikumsleitung kann bei Verdacht auf einen Täuschungsversuch eine mündliche Prüfung der kompletten Gruppe über das Projekt durchführen. Diese mündliche Prüfung beeinflusst maßgeblich die Note bzw. das Bestehen des Praktikums. Bei bestätigten Täuschungsversuchen wird das Praktikum automatisch mit der Note 5.0U bewertet und gilt als nicht bestanden; es wird keine inhaltliche Bewertung der Abgabe vorgenommen. Abgegebener Assemblercode, der von einem Compiler aus einer Hochsprache generiert wurde und nicht explizit als solcher gekennzeichnet ist, wird ebenfalls als Täuschungsversuch gewertet. Abgegebener Code, der von Sprachmodellen stammt, wird ebenfalls als Täuschungsversuch gewertet.

5.4. Nicht eindeutige und Härtefälle

In allen hier nicht oder nicht eindeutig aufgeführten Fällen entscheidet die Praktikumsleitung in Absprache mit Herrn Prof. Dr. Schulz und Herrn Prof. Dr. Wille über den Erhalt des Scheins. Dies gilt insbesondere, falls nach Meinung des Studierenden ein begründeter Härtefall vorliegen sollte. In diesem Fall kann der Studierende dies bei der Praktikumsleitung melden und einen entsprechenden Antrag stellen.

A. Bewertungskriterien

Folgende Listen beschreiben die Verteilung der Punkte auf die Projektbestandteile. *Faktoren* werden mit einem Wert zwischen 0.7–1 bewertet und mit der Gesamtpunktezahl in der jeweiligen Rubrik multipliziert; Abzüge werden danach angerechnet. Je nach Schwere können die Punktabzüge durch Formalitäten auch geringer ausfallen.

Implementierung (22 Pkt.)

Funktionalität (4 Punkte)
- Basisfunktionalität: Korrekte Ergebnisse aller Impl.
- Randfälle gut abgedeckt, sinnvolle Fehlermeldungen.
- Aufgabenspezifische Kriterien erfüllt
- Sinnvolle/erwartete Benutzungsmöglichkeit
- etc.

Art der Implementierung (8 Punkte)
- Kein redundanter Code
- Einhaltung der Calling Convention
- Korrekte Speicherverwaltung
- Keine Segmentation Faults
- Gute und sinnvolle Kommentare
- etc.

Optimierungen (4 Punkte)
- Algorithmische Optimierungen umgesetzt
- SIMD sinnvoll verwendet
- Gegebenenfalls Mikrooptimierungen

Vergleichsimplementierungen (2 Punkte)
- Sinnvolle Vergleichsimplementierungen, die sich zur Evaluation eignen.

Benchmarks (2 Punkte)
- Korrekte Zeitmessung
- Übersichtliche Ausgabe von Ergebnissen
- etc.

Projektbericht (2 Punkte)
- Kurzbeschreibung der Aufgabenstellung
- Auflistung der Vergleichsimplementierungen
- Dokumentation der Messumgebung und Ergebnisse
- Persönliche Anteile der einzelnen Projektmitglieder

Äußere Form (Faktor)
- Gute Inline-Hilfe (`-h`) & Fehlermeldungen
- Sinnvolle & verständliche Codestruktur
- Einrückungen, generelle Lesbarkeit
- etc.

Formalitäten (Abzug)
- (-20) Abgabe kompiliert nicht/unfunktional
- (-10) Aufgabenstellung grob verletzt
- (-4) Falsche Dateinamen, mehrere Binaries, mehrere Code-Versionen in Abgabe

Abschlussvortrag (22 Pkt.)

Problemstellung (3 Punkte)
- Angemessene Wahl der Inhalte
- Verständliche und wissenschaftliche Erläuterung des zu lösenden Problems
- Veranschaulichungen und Beispiele

Lösungsansätze und Optimierungen (3 Punkte)
- Erläuterung der zentralen algorithmischen und technischen Überlegungen
- Abhandlung der Vergleichsimplementierungen
- Überlegungen zu Optimierungen, SIMD, Randfälle, etc.

Korrektheits/Genauigkeitstests (2 Punkte)
- Erklärung der Methodik zur Überprüfung der Korrektheit
- Wahl der Beispiele und mögliche Randfälle

Performanzergebnisse (3 Punkte)
- Erklärung der Benchmarkingmethodik
- Darstellung, Interpretation
- Vergleiche zwischen Implementationen
- Erklärungen zu Laufzeitverhalten
- Sonstige Evaluation der Ergebnisse

Sonstige Foliengestaltung (3 Punkte)
- Verständlichkeit, Unterstützung des Vortrags, Rechtschreibung, Lesbarkeit, Diagramme etc.

Fachliches Verständnis (8 Punkte)
- Antworten zu Theoriefragen
- Fragen zu Projekt, Hardware, Software, ...
- Korrekte Verwendung von Fachbegriffen, präzise Argumentation
- etc.

Formalitäten (Abzug)
- Abgabeformalitäten nicht eingehalten
- Ungleich verteilte Redezeiten je nach Schwere

LEHRSTUHL FÜR RECHNERARCHITEKTUR UND PARALLELE SYSTEME

Grundlagenpraktikum: Rechnerarchitektur

Handreichung zur Projektphase

1 Allgemeines

- Die Praktikumsordnung beschreibt Ablauf, Anforderungen und Bewertungskriterien, daher sollte man diese *gründlich und ggf. auch mehrfach* durchlesen! Wir erwarten, dass die dort beschriebenen Anforderungen vollumfänglich eingehalten werden. Dieses Dokument ist *kein* Ersatz für das Lesen der Praktikumsordnung und Aufgabenstellung.
- Die *Formalitäten* sind vollumfänglich einzuhalten, Formfehler führen meist starke Punktabzüge mit sich. Insbesondere sind auch Dateinamen genau zu beachten.[1]

2 FAQ

1 Wo finde ich die Regeln für das Projekt, seine Bestandteile und die Bepunktung?

Im Projektteil der Praktikumsordnung. Dieses Dokument ist eine zusätzliche Erläuterung und Hilfestellung.

2 Müssen wir ein Assemblerprojekt implementieren?

Das Projekt ist *kein Assembler-Projekt*, eher ein C-Projekt mit optionalem Assembler-Teil.

3 Welcher Teil des Projekts ist der wichtigste?

Die Implementierung ist nicht wichtiger als andere Teile, sie ist ein Mittel zum Zweck, notwendig zur Evaluierung. Die Bewertungskriterien und deren Gewichtung lassen sich auch zur Abschätzung des Umfangs der einzelnen Projektteile heranziehen. Teile mit der gleichen Punktzahl sollten einen ähnlichen Arbeitsaufwand erfordern.

4 Dürfen wir uns die Arbeit aufteilen?

Ja. Sie dürfen sich die Arbeit beliebig aufteilen. Achten Sie jedoch darauf, dass alle Gruppenmitglieder sich gleichermaßen am Projekt beteiligen.

Fachliches Verständnis wird gleichermaßen von *allen* Teilnehmern zu *allen* Teilen des Projekts erwartet.

[1] Dies erleichtert uns die Automatisierung der Bewertung. Jede Abweichung führt Handarbeit mit sich, die *sozusagen* als Punktabzug in Rechnung gestellt wird.

5 Ein Teammitglied antwortet nicht auf Kontaktversuche oder arbeitet nicht mit, was tun wir?

Siehe 16 und die Praktikumsordnung.

6 Worauf müssen wir bei der Abgabe unseres Programmes achten?

Genau *eine* Abgabe (Versionskontrolle mit Git) von genau *einem* Programm (nur eine Executable wird durch `make` generiert). Verschiedene Varianten oder Vergleichsimplementierungen, Tests und Benchmarks sollten über Kommandozeilenargumente wählbar und ausführbar sein.

Das Programm muss auf der Rechnerhalle mit GCC & GAS kompilieren und funktionsfähig sein.

7 Wie soll unsere Implementierung aussehen?

- Lesbaren und übersichtlichen Code schreiben (wenn `main` 1000 Zeilen lang ist, läuft sicherlich etwas falsch). Ggf. ist der Code in mehrere Funktionen sinnvoll aufzuteilen. In dieser Hinsicht auch „DRY“ beachten – don't repeat yourself!
- Das Programm sinnvoll kommentieren:
  - Keine offensichtlichen Kommentare („schließt Datei“, „erhöhe um 1“, etc.)
  - Mit sinnvollen Variablennamen sollte der Code größtenteils verständlich sein
  - Annahmen, komplexere Codeteile, etc. verständlich und präzise dokumentieren
- Globale (schreibbare) Variablen vermeiden. Es gibt eigentlich wenig Gründe, globale Variablen zu verwenden; diese erschweren aber das Verständnis und die Benutzbarkeit von Code, weil dies ein „unsichtbarer“ Zustand ist – gerade bei Programmen mit mehreren Threads. `static const` kann man in C auch innerhalb einer Funktion definieren.
- Stack-Allokationen bevorzugen, wenn Größe hinreichend klein und konstant oder beschränkt ist. Bei potentiell größeren Allokationen sollte auf dem Heap alloziiert werden.
- Auf konsistenten Code-Stil (Indents, Spaces bei Klammern, Position von Braces, Variablennamen, etc.) achten.[2]
- Deutschen oder englischen Code und Kommentare schreiben.

[2] Zum konsistenten Formatieren von C-Code kann z.B. das Tool „ClangFormat“ hilfreich sein.

- Potenzielle Fehler sind auch Fehler, beispielsweise wenn der Erfolg von `malloc` nicht geprüft wird.[3]
- Keine Segmentation Faults oder Buffer-overflows, etc. – *niemals*!
  - Randfälle in Nutzereingaben abfangen, etc.
  - Überprüfen Sie Ihren Code mit Analysetools (Valgrind, perf, sanitizer).
- Pointer-Funktionsparametern korrekt (und sinnvoll) mit `const` (und `restrict`) annotieren.
- *Undefined behavior* vermeiden.[4]

8 Worauf müssen wir beim Rahmenprogramm achten?

- Beachten Sie das in der Angabe spezifizierte Kommandozeileninterface. Dies muss *genau* eingehalten werden. Ihr Programm muss alle dort genannten Optionen/Parameter unterstützen und nur mit den dort genannten Optionen/Parametern verwendbar sein. Möchten Sie weitere Optionen/Parameter hinzufügen, müssen Sie für diese sinnvolle default Werte definieren.
- Programm mit Hinblick auf Verwendung, Fehlermeldungen, etc. benutzerfreundlich schreiben:
  - `-h|--help` soll Nutzungshilfe ausgeben und erwartete Kommandozeilenargumente erklären. Von (GNU-)Konventionen für Programmargumente sollte nicht abgewichen werden.
  - Fehlermeldungen sollen erkennbar machen wo das Problem ist (z.B. „`Could not open "mandrill.bmp": No such file or directory`“). Für die Ausgabe von Fehlermeldungen muss `stderr` genutzt werden. In C ist hierfür beispielsweise die Funktion `perror` empfehlenswert.
- Kommandozeilenargumente, bevorzugt mit `getopt`, zur Konfiguration verwenden und `stdin` vermeiden, sofern dies nicht von der Aufgabenstellung explizit gefordert wird. Diesbezüglich ist evtl. auch der folgende Link von Interesse: `http://sekrit.de/webdocs/c/beginners-guide-away-from-scanf.html`

[3] Im Rahmen des Praktikums genügt es meist, bei Speicherknappheit die Programmausführung mit einer aussagekräftigen Fehlermeldung abzubrechen; allgemein ist dies aber natürlich kein Grund, die Ausführung einzustellen: Beispielsweise könnte ein anderer, ggf. langsamerer, Algorithmus mit weniger Speicherbedarf verwendet werden, oder es werden mehr I/O-Operationen durchgeführt (Streaming).

[4] Während dies im Praktikum nicht explizit bewertet wird, kann sich das Programmverhalten trotzdem zwischen unterschiedlichen Compilerversionen oder gar Kompilierungsoptionen unterscheiden. Beispiele für *undefined behavior*: `NULL`-Pointer-Dereferenzierung, Signed-Integer Overflow, Dereferenzieren von Objekten mit unterschiedlichen Datentypen (z.B. nach Casts!), uninitialisierte Variablen, oder Out-of-Bounds-Access.

- Auf sinnvolle Implementierung und Konfigurierbarkeit der Tests und Benchmarks achten. Insbesondere muss die Implementierung, die Eingabe und wenn angemessen eine Anzahl der Wiederholungen wählbar sein. Es muss zwischen einem funktionalen und einem Benchmark-Modus unterschieden werden.

9 Wie soll der Projektbericht aussehen?

- Der Projektbericht soll dabei helfen, schnell einen Überblick über Ihr Projekt, den Code, die Performanzanalyse und die Projektarbeit zu bekommen.
- Maximal 50% davon sollten in einem gemeinsamen Teil die Aufgabenstellung, Ihre Lösungsansätze und das Benchmarking beschreiben, im Rest des Berichtes beschreiben alle Gruppenmitglieder kurz den eigenen Anteil an der Projektarbeit.
- Es kann Sinn machen, die Messumgebung für das Benchmarking hier zu dokumentieren.
- Der Projektbericht is als Markdown-Datei zu verfassen.
- Maximal 400 Wörter. Fassen Sie sich kurz!

10 Wie geben wir die Vortragsfolien ab?

Folien sind auch bis zur Deadline unter dem *richtigen Dateinamen* abzugeben.

11 Wie soll der Vortrag aussehen?

- Der Vortrag sollte möglichst interessant und verständlich sein, insbesondere für Personen, die sich zuvor *nicht* mit dem Thema auseinandergesetzt haben. Ihre Kommilitonen sollten dem Vortrag folgen können.
- Der Fokus sollte auf Ansätzen und Ergebnissen liegen. Implementierungsdetails, Code, Programmaufrufe, etc. sind meist nicht interessant.
- Sprache: Deutsch.

12 Welchen Inhalt soll der Vortrag haben?

Der Vortrag soll die folgenden Bestandteile und Inhalte haben:

Problemstellung Die Problemstellung nachvollziehbar erklären, wenn sinnvoll auch mit Beispielen.

Lösungsansatz

- Erklären von Konzepten und Ansätzen. Dabei allerdings nicht auf Implementierungsdetails oder irrelevante Bestandteile (z.B. Parsen der Kommandozeilenargumente) eingehen und auf ein angemessenes Abstraktionsniveau achten.
- Beschreiben von großen und/oder nicht direkt ersichtlichen Optimierungen (z.B. SIMD-Verarbeitung), bzw. Argumentieren warum diverse Optimierungen nicht möglich waren. Triviale Optimierungen *nicht* erwähnen.
- Nicht den Code Zeile für Zeile erklären.
- Konzepte sollten mithilfe von Grafiken veranschaulicht und verdeutlicht werden.

Korrektheit/Genauigkeit

- Abhängig von Aufgabenstellung entweder Korrektheit *oder* Genauigkeit analysieren – und die Wahl ggf. begründen!
- Achten Sie auf folgendes bei der Durchführung Ihrer Analyse:
  - Es soll insbesondere (auch) die konkrete *Implementierung* auf ihre Korrektheit/Genauigkeit analysiert werden.
  - Achten Sie auf eine wissenschaftliche Vorgehensweise und dokumentieren Sie Ihr Vorgehen. Insbesondere sollten angemessene Vergleiche/Vergleichsimplementierungen und Eingabebereiche gewählt werden.
- Relevante Beispiele zeigen, die die Funktionalität und Korrektheit oder Genauigkeit demonstrieren und eventuelle Aussagen unterstützen. Manchmal kann es sinnvoll sein, auch Randfälle als Beispiele anzugeben.

Performanzanalyse

- Verschiedene Implementierungen und Varianten miteinander vergleichen. Den Einfluss getroffener Optimierungen auf die Performanz klar aufzeigen und analysieren – auch wenn keine Verbesserung erreicht wurde (oder sich die Performanz gar verschlechtert hat).
- Eine gute Performanzanalyse vergleicht – wenn sinnvoll – auch verschiedene algorithmische Ansätze.
- Achten Sie auf folgendes bei der Durchführung Ihrer Analyse:
  - Hier soll insbesondere (auch) die konkrete *Implementierung* auf ihre Performanz analysiert werden.
  - Die Hinweise aus den Tutorübungen bezüglich des Benchmarkings beachten.

  - Die Vorgehensweise und Ergebnisse sind *wissenschaftlich* zu dokumentieren und zu analysieren. Die Testumgebung (relevante Hardware, Software, ggf. Vergleichsprogramme und weitere relevante Parameter) ist genau zu dokumentieren. Insbesondere sollten Ergebnisse nicht nur genannt, sondern auch mithilfe der gesammelten Daten begründet werden. Falls nötig mehr Daten erheben.

- Ergebnisse wenn möglich grafisch darstellen. Es sollte sich allerdings auf die Darstellung von relevanten oder interessanten Ergebnissen beschränkt werden.
- Unerwartete Ergebnisse sollten begründet und diskutiert werden.
- Bei Vergleichen von Implementierungen ist es für die Nachvollziehbarkeit ratsam, präzise zu spezifizieren, was verglichen wird. Anstatt "Assembler vs. C" ist zum Beispiel "SIMD vektorisierte Assemblerimplementierung vs. Naive C Implementierung" aussagekräftiger. Gegebenfalls können Angaben zur Toolchain (z.B. GCC 10.2 mit Optimierungsoptionen `-O3 -fno-unroll-loops`) hilfreich sein.
- Die Messumgebung und Messmethodik muss genau dokumentiert werden. Diese Dokumentation muss aber nicht im Detail Teil der Präsentation sein. Sie können Ihre Messmethodik- und Umgebung auch anderweitig dokumentieren und nur die wichtigsten Informationen im Vortrag selbst nennen. Auch in diesem Fall sollten Sie Fragen zu Ihrer Messmethodik beantworten können.

Zusammenfassung Knappe Zusammenfassung des Projekts sowie ggf. ein Ausblick.

13 Wie sollen die Folien für den Vortrag aussehen?

- Auf Lesbarkeit achten (Schriftgröße, Farben, Kontrast), insbesondere bei Grafiken.
- Seitenverhältnis 16:9.
- Folien nicht überfüllen (max. 8 Zeilen Text pro Folie, keine vollständigen Sätze).
- Folien nummerieren! So können Zuhörende sich einfach notieren zu welchen Folien sie Fragen hatten.
- Nicht zu viele Folien; mit min. einer Minute Vortrag pro Folie planen.

14 Werden nach dem Vortrag Fragen gestellt?

Ja. Nach dem Vortrag werden allen Gruppenmitgliedern Fragen gestellt werden. Diese können sich um Praktikumsinhalten, sowie Ihr Projekt drehen.

15 Was ist bei der Optimierung wichtig?

Konzeptionelle und algorithmische Optimierungen sind wichtiger als Mikrooptimierung *aber*: kein unnötig ineffizienter Code. Wichtiger als Optimierungen ist, Performanzunterschiede im Vortrag (korrekt) begründen zu können.

16 Wir haben ein Problem, an wen wenden wir uns?

- Fragen zum Praktikumsinhalt ⇝ Selbst herausfinden
- Programm kompiliert nicht ⇝ Fehlermeldung lesen
- Programm funktioniert nicht (richtig) ⇝ Debuggen
- Unklarheit Praktikumsordnung ⇝ Tutor oder öff. Zulip
- Unklarheit Aufgabenstellung ⇝ Tutor
- Gruppenmitglied springt ab ⇝ Tutor
  - Aufgabenverringerung nur nach Anfrage
- Gruppenmitglied arbeitet nicht/meldet sich nicht ⇝ Tutor
- Ausnahmeanträge ⇝ ÜL
- Alles andere ⇝ Erstmal zum Tutor

Für Emails verwenden Sie bitte ggf. eine Ihrer TUM-Emailadressen. Geben Sie bitte immer Ihre Teamnummer an.

3 Tipps

- Die Bewertung der Projektbestandteile erfolgt grundsätzlich durch Abzüge. Wir raten daher, *Qualität über Quantität* zu priorisieren. Lieber weniger „Features“ in das Programm einbauen, dafür die vorhandenen (möglichst) fehlerfrei implementieren.
- Zunächst: einfachen Ansatz in C implementieren
  - Vollständig funktionsfähiges (einfaches) Rahmenprogramm
  - Erste Referenzimplementierung zum Testen von Korrektheit und Performanz

  dann erst die optimierte Hauptimplementierung umsetzen.
- Zeitmanagement: Es empfiehlt sich dringend, *frühzeitig anzufangen* und die erforderliche Zeit nicht zu unterschätzen. Die meisten Resultate benötigen nach der initialen Erstellung noch mehrere Iterationen zur Qualitätsverbesserung. Ein guter wissenschaftlicher Vortrag braucht (viel) Zeit, gerade wenn man zu Beginn des

Studiums noch wenig Erfahrung hat. Für den (unwahrscheinlichen) Fall, das man doch eher fertig werden sollte, hat man auch später mehr Zeit sich auf andere Prüfungen vorzubereiten.

4 Überblick / Checkliste zur Abgabe

Ergänzend noch einmal die wichtigsten formalen Aspekte, die bei der Abgabe beachtet werden müssen, zusammengefasst.[5]

4.1 Allgemein

- **Vor der Abgabe Praktikumsordnung, Handreichung und Aufgabenstellung noch einmal genau durchlesen und überprüfen, ob das Projekt den Anforderungen entspricht.** Dazu sollte man das Repository einmal neu herunterladen (`clone`) und prüfen, dass alles passt (Programm kompiliert, alle Dateien vorhanden und am richtigen Ort mit exakt richtigem Namen, etc.).
- Änderungen in letzter Minute, die nicht mehr getestet werden können, sollten vermieden werden.
- Die Abgabe erfolgt auf dem `main`-Branch des Git-Repositorys.
- Die folgende Ordnerstruktur Ihres Git-Repositorys **muss** eingehalten werden:

Projektbericht.md:	Ihr Projektbericht
Implementierung/:	Ihre Implementierung
Implementierung/Makefile:	Makefile für Ihre Implementierung, welches durch einen Aufruf von *make* Ihre Implementierung kompiliert
Vortrag/Vortrag.pdf:	Folien für Ihre Abschlusspräsentation

4.2 Implementierung

- Die Implementierung muss mit *make* auf der Referenzplattform des Praktikums (lxhalle) kompilieren. Es soll keine Binary abgegeben werden.
- *make* soll nur eine ausführbare Datei erzeugen.

4.3 Vortrag

- Stellen Sie sicher, dass 16:9 als Folien-Format verwendet wird.
- Stellen Sie sicher, dass die Folien nummeriert sind.
- Achten Sie auf ausreichenden Kontrast und Schriftgröße. (Die Qualität der Beamer kann schlechter als erwartet sein).

[5] Diese Liste ist u.U. nicht vollständig!